Università degli Studi di Messina
Dipartimento MIFT - Corso di Laurea Triennale in Informatica (L-31)

Laboratorio di Reti e Sistemi Distribuiti

HANDS-ON 10:
Grafi di Precedenza e Parallelismo

Massimo Mantineo
Matricola: 541924

Messina, A.A. 2024/2025

Indice

1 Problem Statement **2**

2 Stato dell'Arte **2**

3 Metodologia ed Implementazione **2**
3.1 Analisi del Grafo (DAG) 2
3.2 Sincronizzazione in C 2
3.3 Istruzioni di Esecuzione 2

4 Risultati **3**
4.1 Validazione Numerica 3
4.2 Confronto Prestazionale 3

5 Conclusioni e Sviluppi Futuri **3**

1 Problem Statement

L'Hands-On 10 (HO10) affronta la formalizzazione matematica e l'implementazione software della scomposizione di un calcolo algebrico tramite i **Grafi di Precedenza**. L'obiettivo è risolvere l'equazione $y = (2*6)+(1+4)*(5-2)$ identificando il massimo grado di parallelismo estraibile. Il sistema deve garantire che le operazioni dipendenti vengano eseguite solo dopo il completamento dei propri predecessori, confrontando l'efficacia della sincronizzazione deterministica in linguaggio C rispetto al modello interpretato di Python.

2 Stato dell'Arte

La progettazione di algoritmi paralleli si basa su criteri formali di indipendenza e limiti teorici di scalabilità.

3 Metodologia ed Implementazione

L'approccio prevede la mappatura dell'equazione su un grafo orientato aciclico (DAG).

3.1 Analisi del Grafo (DAG)

L'espressione viene scomposta in tre livelli di esecuzione (L_0, L_1, L_2):

3.2 Sincronizzazione in C

Viene utilizzata la libreria `pthread` per gestire il ciclo di vita dei task. Il recupero dei valori avviene tramite il casting del puntatore di ritorno nella `pthread_join`.

Snippet Sincronizzazione Livelli

```c
pthread_create(&tA, NULL, task_A, NULL);
pthread_create(&tB, NULL, task_B, NULL);
pthread_create(&tC, NULL, task_C, NULL);
// Barriera di sincronizzazione manuale
pthread_join(tB, (void**)&resB);
pthread_join(tC, (void**)&resC);
// Ora posso lanciare il task D (Livello 1)
```

3.3 Istruzioni di Esecuzione

Compilazione del risolutore di grafi:

Build

```
$ gcc graph_solver.c -o solver
```

4 Risultati

La validazione è stata condotta verificando l'integrità del calcolo e l'ordine di esecuzione dei thread.

4.1 Validazione Numerica

Entrambe le implementazioni (C e Python) hanno restituito con successo il valore atteso $y = 27$. I log di sistema confermano che il thread responsabile dell'operazione $y = A + D$ è rimasto in stato di *waiting* finché i predecessori non hanno rilasciato le risorse.

4.2 Confronto Prestazionale

L'ispezione con `htop` ha mostrato un utilizzo bilanciato di 3 core durante la fase L_0 in C, mentre in Python l'overhead del context switching per il GIL ha limitato l'efficienza computazionale effettiva.

5 Conclusioni e Sviluppi Futuri

L'Hands-On 10 ha dimostrato che la scomposizione in grafi di precedenza è il metodo più rigoroso per strutturare la computazione parallela, garantendo la coerenza dei dati (*data consistency*) in sistemi distribuiti.
Sviluppi futuri:

- **Analisi Prestazionale**: Studio dellímpatto della latenza di sistema sulla scalabilità dellárchitettura.
- **Ottimizzazione delle Risorse**: Refactoring del codice per minimizzare lúso della CPU in contesti ad alta concorrenza.